

High Performance Programming - Assignment 3

Hugo LINDSTRÖM
William OLSSON
Arnaud GAILLARD
Sarafroz BEKMURODOVA

2026-02-16

1 The Problem

The task is to implement a simple simulation of N particles interacting through gravity from some initial conditions.

2 The Solution

Our solution is based on the symplectic Euler's method. The idea of the symplectic Euler's method is to use the last state to approximate the next [1]. To find the next position, we look at the previous position and add the object's velocity multiplied by the time between the last and current step. If we used the regular Euler method [2], also called the explicit Euler method, we would use the velocity from the previous step. In the symplectic Euler method, we first calculate the approximate velocity at the current time step and use it to update the positions of our particles.

We created one main struct called Particle:

```
struct Particle {  
    double x_pos;  
    double y_pos;  
    double mass;  
    double x_velocity;  
    double y_velocity;  
    double brightness;  
};
```

We use two global arrays:

particles	current state of all particles
temp_particles	next state — newly calculated positions & velocities

Both are allocated with malloc for n particles. We also have a small epsilon = 0.001

The code is split into a few clear functions:

readArguments()	reads command-line inputs: N , filename, nsteps, delta_time, graphics flag
readFile()	allocates memory and loads the initial particle data with fread
step()	computes one time step (forces, accelerations, updates)
write_file()	saves final particle data to "results.gal" with fwrite
main()	checks usage, reads arguments & file, runs the simulation loop, writes output

the time between the last and current step, multiplied by t

Inside step() we do classic N -body gravity:

```

For each particle i (from 0 to n-1):
    Reset temp_particles[i].x_velocity = 0
    Reset temp_particles[i].y_velocity = 0

Set G = 100.0 / n

For each particle i (from 0 to n-1):
    mass_i = particles[i].mass

    For each particle j (from i+1 to n-1):    // j > i, using Newton's 3rd law
        mass_j = particles[j].mass

        dx = particles[i].x_pos - particles[j].x_pos
        dy = particles[i].y_pos - particles[j].y_pos

        distance = sqrt(dx*dx + dy*dy)
        factor = G / (distance + epsilon)^3

        // Force contribution
        force_x = factor * dx
        force_y = factor * dy

        // Accelerations using Newton's 3rd law
        accel_i_x = -force_x * mass_j
        accel_i_y = -force_y * mass_j

        accel_j_x = force_x * mass_i
        accel_j_y = force_y * mass_i

        // Accumulate velocity changes in temp_particles
        temp_particles[i].x_velocity += delta_time * accel_i_x
        temp_particles[i].y_velocity += delta_time * accel_i_y

        temp_particles[j].x_velocity += delta_time * accel_j_x
        temp_particles[j].y_velocity += delta_time * accel_j_y

// Apply accumulated velocity changes and update positions
For each particle i (from 0 to n-1):
    particles[i].x_velocity += temp_particles[i].x_velocity
    particles[i].y_velocity += temp_particles[i].y_velocity

    particles[i].x_pos += particles[i].x_velocity * delta_time
    particles[i].y_pos += particles[i].y_velocity * delta_time

If graphics enabled:
    draw_galaxy()
}

```

Then we swap the two arrays. The gravitational constant is set to $G = \frac{100.0}{n}$ so the forces stay reasonable no matter how many particles we have.

Why forward Euler?

It's the simplest integrator — easy to code and understand. Requires very few temporary

variables. Better methods like Runge-Kutta 4 would keep energy more stable, but they're more complicated and probably not required here. Double buffering (two full arrays: particles + temp_particles, swap pointers after each step). Alternatively we can use single array + separate force/acceleration arrays. But double buffering is the safest and clearest way, also pointer swap is fast.

3 Performance and discussion.

Let us start by making a make test_performance rule that runs our code with the following input time ./galsim 01000 ./input_data/ellipse_N_01000.gal 100 0.00001 0 as well as time ./galsim 10000 ./input_data/ellipse_N_10000.gal 100 0.00001 0.

The code is compiled using gcc with the following flags: -O3 -Wall -Wextra -pedantic -g -march=native -ffast-math -ftree-vectorize -fopt-info-vec.

3.1 Initial performance

Using the naive algorithm that is of complexity $O(N^2)$, we get a runtime of 0.48s when the input is of size 1000 and 47s on input size 10000.

3.2 Using Newton's Third Law

We are calculating the same force multiple times. This is because we first calculate the force that particle j exerts on particle i, then the force that particle i exerts on particle j. Using Newton's third law, we can combine the two calculations into a single step. The new algorithm has a time complexity of $O(N * (N - 1))$, which simplifies to $O(N^2)$.

3.2.1 Performance Gain

We get a runtime of 0.31s for an input of size 1000 and 29s for an input of size 10000. This aligns with our time complexity of $O(N^2)$.

3.3 Removing Redundant Divisions

The formula for gravitational force contains the mass of each particle, but the formula for acceleration immediately divides the result by the particle's mass. We can simplify the operations to require only one multiplication instead of two multiplications and one division.

3.3.1 Performance Gain

We get a runtime of 0.24s for an input of size 1000 and 23s for an input of size 10000. Again, this aligns with our time complexity of $O(N^2)$.

3.4 Final Algorithms Performance Growth

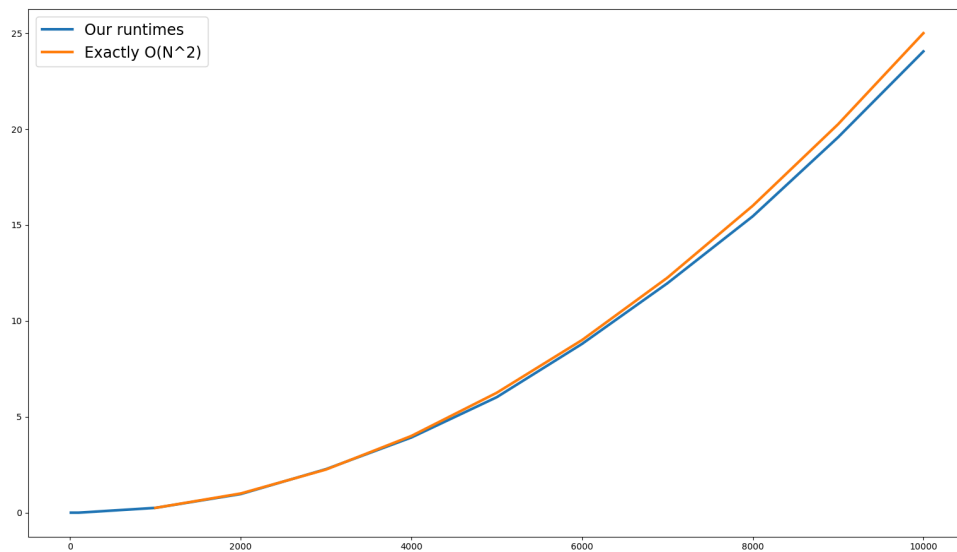


Figure 1: Runtime of our program (in seconds) as the input size N increases (number of particles) vs the expected $O(N^2)$ runtime.

References

- [1] “Assignment 3.” Accessed: Feb. 10, 2026. [Online]. Available: <https://uppsala.instructure.com/courses/109137/assignments/373458>
- [2] “Euler method.” Nov. 17, 2025. Accessed: Feb. 10, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Euler_method&oldid=1322588975

All the references you have used for code and theory. If we find that you have copied code or code sections without a proper reference, it will be considered plagiarism.